IIITA Timetabling System

A thesis submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

By:-	*Enrollment No.*
ESHANT	IIT2023198
RAHUL ROY	IIT2023196
LAKAVATH PEER SINGH	IIT2023197

Under the Supervision of

DR. GAURAV SRIVASTAVA

to the

DEPARTMENT OF INFORMATION TECHNOLOGY

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
ALLAHABAD

Wednesday 7th May, 2026

भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad
An Institute of National Importance by Act of Parliament
Deoghat Jhalwa, Prayagraj 211015 (U.P.) India
Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CERTIFICATE

It is certified that the work contained in the thesis titled “IIITA Timetabling System” by Eshant, Rahul Roy, Lakavath Peer Singh has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Dr. Gaurav Srivastava
Department of Information Technology
IIIT Allahabad

भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad
An Institute of National Importance by Act of Parliament
Deoghat Jhalwa, Prayagraj 211015 (U.P.) India
Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CANDIDATE DECLARATION

We, Eshant (IIT2023198), Rahul Roy (IIT2023196), and Lakavath Peer Singh (IIT2023197), hereby certify that the thesis titled "IIITA Timetabling System" has been submitted by our in partial fulfillment of the requirements for the Degree of Bachelor of Technology in the Department of Information Technology at the Indian Institute of Information Technology, Allahabad.

We acknowledge that plagiarism includes:

1. Reproducing another person's work (fully or partially) or ideas and presenting them as our own.
2. Copying or paraphrasing another person's work without appropriate attribution.
3. Engaging in literary theft by reproducing a unique literary construct without crediting its source.

We affirm that due credit has been given to all original authors and sources through proper citations for words, ideas, diagrams, graphics, computer programs, experiments, results, and websites that are not our own contributions. Direct quotations have been appropriately marked, and sources have been properly referenced. Furthermore, we declare that no portion of our work is plagiarized. In the event of a plagiarism claim, we accept full responsibility for the contents of this thesis. We acknowledge that our supervisor may not be able to verify the originality of every aspect of our work.

Place: **IIITA** Date: **07/05/2026**

Eshant (IIT2023198)

Rahul Roy (IIT2023196)

Lakavath Peer Singh (IIT2023197)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, ALLAHABAD

IIITA Timetabling System

ABSTRACT

University timetabling is a complex problem, or rather, it is an NP-hard problem. Even small campuses have hundreds of constraints to schedule for: courses, professors, rooms, and groups of students. At IIIT Allahabad, the timetables were generated manually in spreadsheets, which was time-consuming, error-prone, and not scalable. In this thesis, we created the ``IIITA Timetabling System", a web-based solution to automate and digitise the entire timetable scheduling pipeline.

We implemented a heuristic-based Excel parser that can process raw, unstructured timetable Excel spreadsheets, extracting information such as subject codes, faculty names and room numbers, and loads them into a cloud-based PostgreSQL database using Supabase.

The main contribution is an automated timetable scheduler based on a (1+1) Evolution Strategy (ES). We represent a timetable as a chromosome with Gene decisions (day, time-slot, room) and evaluate candidate solutions using a fitness function with 10 hard constraints (room / professor / group clashes, break boundary limits, labs, elective synchronisation, etc.) and 3 soft constraints (minimising gaps for students, maximising room occupancy and keeping professors in one side of the day). The algorithm adjusts step sizes using Rechenberg's classic 1/5th rule, restarts when trapped in local optimums and can be initialised with a given timetable. To prevent clashes across semesters, we populate already scheduled sessions from published timetables.

Since no solver can anticipate every conceivable real-world preference, we also provide a drag-and-drop Timetable Editor. Administrators can visually rearrange the sessions, with conflicts displayed in real-time, and LNS (Large Neighbourhood Search) auto-fix with one click. A Master View displays slots from all published timetables so conflicts across semesters are instantly apparent.

In daily life, however, the system allows customized views for each stakeholder, from the students' view of their individual schedule to professors' schedules and a master occupancy map for the entire university campus, plus the ability to find available rooms using our Free Room Finder module. In addition, we developed an extensive Intelligent Generator to guide administrators step-by-step.

React 19, TypeScript, and Tailwind CSS form the technology stack used throughout the application, making it very flexible and modern in design. The use of the Google Sheets API makes data export elegant, while PDF creation is possible locally as well. In the context of this thesis, topics such as the theoretical framework of the timetabling problem, the technology stack and architecture, evolutionary algorithm with constraints, and interactive editor components are discussed.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Each academic semester, the university faces the same challenging problem. How do you allocate hundreds of classes, a number of lecturers, and a fixed set of class rooms to a semester-wide timetable? This problem is known as the University Timetabling Problem (UTP) and it has been proven NP-hard, i.e., that there is no efficient algorithm for solving the problem optimally. This chapter will give a brief overview of the IIITA Timetabling System, describe its rationale, and outline the remaining chapters in this thesis.

1.1 Background and Motivation

An academic timetable involves assigning lectures, tutorials, and laboratory classes to a particular time slot, venue, and lecturer, while taking into account the limitations imposed by the situation. There are some constraints that are rigid, in that it is not possible to double book the lecturer or run two classes in one room at the same time. On the other hand, there are certain soft constraints which include having the minimum amount of gap between lectures, and the professors prefer lecturing in one half of the day only.

IIIT Allahabad is one among the many colleges in India where the scheduling process remains manual. Timetables are created manually using Microsoft Excel, and conflicts are manually resolved by the academic coordinators. Even though Microsoft Excel is highly

customizable and flexible, it does not have any sense of the meaning of a “professor” or “room”. It is up to the manual efforts of the people preparing the timetables to make sure that such problems are avoided.

The issues do not stop even after the timetable is released. Using a static PDF or Excel file will result in the need to send out the whole document every time there is a change, such as a classroom change or an instructor change. The students will have to work with outdated versions, and no one will be able to answer basic queries such as "What classrooms are available at 2 PM on Wednesday?"

These annoyances were what pushed us to develop the IIITA Timetabling System. We sought a unified solution that would become the ultimate authority on all scheduling information. This system should be capable of automatically generating conflict-free timetables, allowing the administrators to adjust them interactively, and providing individualized timetables for each stakeholder.

1.2 Problem Statement

The underlying issue we sought to address is simple: The manual creation and management of timetables via spreadsheets at IIITA is inefficient, error-prone, and inherently inflexible. Specifically, the existing process suffers from:

- **Unstructured Data:** Our schedule is created in excel sheets with merged cells, non-matching names, and relations that are beyond any computer program’s comprehension.
- **Manual Conflict Resolution:** To check whether there are problems like clash of classrooms, unavailability of certain teachers or courses, one needs to do a whole

process manually, hence increasing the possibility of error.

- **No Automated Generation:**Currently, there is no system to automatically create a non-conflict timetable according to the needs of our institution.
- **One-Size-Fits-All Views:** It makes no sense to make a timetable which will not satisfy everyone. Students need to search in this timetable, and teachers cannot quickly find their schedules.
- **Resource Management:** Administrators lack a simple way of knowing which classrooms are vacant at certain times and how effectively our resources are being used.
- **Static Distribution:** After publishing the timetable in pdf form, updating it means distributing it all over again.

Thus, our objective was to develop a system which could help close the chasm between unstructured spreadsheet formats and structured data, schedule creation, editing tools, and provide role-based visualization of the timetable.

1.3 Objectives of the Study

We set ourselves seven concrete objectives:

1. **Heuristic Data Ingestion:** Design an engine that will parse IIITA's complicated Excel timetable, extract metadata (course subjects, instructors, class venues), merge cells, and normalise into a relational schema.
2. **Real-Time Clash Detection:** Design a set of algorithms for detecting conflicts related to time, room availability, instructor workload, and student sections immediately upon importing timetable data.

3. **Role-Based Views:** Design role-based UI interfaces for different actors in the process including:
    - Student Interface with schedule filtering options and electives.
    - Professor Interface with personal schedule only visible to the professor.
    - Admin Interface with master schedule, free room finder, and other administrative tools.
4. **Export and Reports:** Provide users with options to export schedule as PDF, as well as providing Google Sheets support for administrative purposes.
5. **Automated Timetable Generation:** Implement a $(1 + 1)$ Evolution Strategy solver that evaluates candidates against 10 hard constraints and 3 soft constraints, adapts its step size via the 1/5th success rule, supports warm-starting from existing schedules, and handling inter-semester constraints through locked sessions.
6. **Interactive Timetable Editor:** Develop a graphical drag-and-drop editor for manual timetable optimisation with instant conflict detection, room and professor modification capabilities, and one-click Large Neightbourhood Search (LNS) auto-resolution for any remaining conflicts.
7. **Intelligent Generator:** Create a guided, multi-step workflow that guides administrators through cluster selection, home room mapping, professor assignment, and solver configuration.

1.4 Scope and Limitations

The scope of the project is to implement all phases of the IIITA Timetabling System, including the React + TypeScript front end, the PostgreSQL back end with Supabase, and the browser-based execution of the (1+1)-ES. Even though the system has been designed specifically for the timetable of IIIT Allahabad, it is still extensible for other colleges.

That said, there are a few important limitations to keep in mind:

- **Parser Sensitivity:** Our Excel parser relies on heuristic rules tuned to IIITA's template. Major deviations from that format may require tweaking the parsing logic.
- **Single-Threaded Solver:** The solver runs on the browser's main thread using batched `setTimeout` calls. This strategy will work well with the present input size, though in case of very large inputs it may become a performance bottleneck.
- **Heuristic Optimality:** Evolution Strategies are meta-heuristics. They guarantee feasibility by providing solutions with zero hard constraint violations, though there is no assurance of optimality for soft constraints based solutions. This depends upon the number of generations.

1.5 Organization of the Thesis

The rest of this thesis is organised as follows:

Chapter 2: Literature Review will provide a review of current researches into university timetabling, including the basic theories of UTP as an NP-hard problem and current algorithmic solutions in literature.

Chapter 3: Technologies and Frameworks will present our choice of technology stack for this project, i.e., React 19, TypeScript, Tailwind CSS, Vite, and Supabase, along with the reasons for their adoption.

Chapter 4: System Architecture and Design will detail the database schema, the data ingest pipeline, the solver architecture, and finally, the client-side routing of components.

Chapter 5: Implementation and Results will form the core of this thesis, where the implementation details of the Excel parser, clash detection algorithms, (1+1)-ES solver and constraint system, drag-and-drop editor and auto-LNS fix, and the intelligent generator will be described.

Chapter 6: Conclusion and Future Work sums up what we achieved, reflects on the challenges we faced, and outlines future directions such as parallelised solvers and hybrid meta-heuristics.

Chapter 2

Literature Review

The challenge of creating conflict-free, optimal schedules for academic institutions has been a subject of extensive research for decades. This chapter reviews the existing literature surrounding the University Timetabling Problem (UTP), categorizes the various algorithmic approaches proposed by researchers, examines the evolution of software architectures used in timetable management systems, and identifies the gaps in existing solutions that this project seeks to address.

2.1 The University Timetabling Problem (UTP)

Academic timetabling is formally defined as the process of assigning a set of events (such as lectures, exams, or meetings) to a set of resources (such as time slots, rooms, and faculty members) subject to a multitude of constraints [**schaerf1999survey**]. The complexity of this task is heavily dependent on the size of the institution, the flexibility of the curriculum, and the rigidity of the constraints.

2.1.1 Constraint Satisfaction and Complexity

In computational complexity theory, the University Timetabling Problem is generally classified as NP-hard [**even1975complexity**]. This implies that as the number of variables

(classes, rooms, professors) increases, the time required to find an optimal solution grows exponentially, making brute-force search algorithms entirely unfeasible for real-world applications.

Constraints within the UTP are universally divided into two categories:

- **Hard Constraints:** These are non-negotiable conditions that must be satisfied for a timetable to be considered feasible. Examples include:
  - A room cannot host more than one class simultaneously.
  - A professor cannot teach two different classes at the same time.
  - A student section cannot be scheduled for two different lectures simultaneously.
  - The seating capacity of an assigned room must meet or exceed the number of students enrolled in the class.
- **Soft Constraints:** These represent preferences that enhance the quality of the timetable but are not strictly necessary for its execution. A penalty score is usually assigned when soft constraints are violated, and the goal of optimization algorithms is to minimize this score. Examples include:
  - Minimizing the number of free periods (gaps) in a student's daily schedule.
  - Ensuring faculty members do not have heavily fragmented schedules across the week.
  - Assigning classes to rooms preferred by the instructor.
  - Grouping specific core subjects together sequentially.

2.2 Algorithmic Approaches to Timetabling

Over the years, researchers have proposed a vast array of techniques to solve the UTP, ranging from classical mathematical modeling to modern meta-heuristic algorithms.

2.2.1 Exact Methods

Early approaches focused on exact mathematical methods, primarily Integer Linear Programming (ILP) and Graph Coloring. In a Graph Coloring model, events are represented as nodes, and edges are drawn between events that conflict (e.g., they share the same student group or professor). The goal is to "color" the nodes—where each color represents a distinct time slot—using the minimum number of colors such that no two connected nodes share the same color [**de1985graph**]. While theoretically sound, exact methods struggle significantly with the high dimensionality of real-world timetabling problems, often failing to find solutions within acceptable computation times when soft constraints are introduced.

2.2.2 Heuristic and Meta-Heuristic Algorithms

Due to the computational limitations of exact methods, the focus of the research community shifted heavily towards heuristic and meta-heuristic approaches in the late 1990s and 2000s. These methods do not guarantee finding the absolute optimal solution but are highly effective at finding "good enough" (feasible and high-quality) solutions within reasonable timeframes.

- **Genetic Algorithms (GA):** Inspired by natural evolution, GAs represent potential timetables as "chromosomes." Through processes of selection, crossover (combining parts of two schedules), and mutation (randomly altering a scheduled event),

populations of timetables evolve over generations to minimize constraint violations [**burke1996genetic**].

- **Simulated Annealing (SA) and Tabu Search:** These local search techniques iteratively make small changes to a complete schedule. Tabu search uses a memory structure (a "tabu list") to prevent the algorithm from revisiting recently evaluated schedules, thereby avoiding getting trapped in local optima [**glover1997tabu**]. Simulated Annealing mimics the metallurgical process of annealing, occasionally accepting "worse" solutions early in the process to explore a broader search space before gradually "cooling" down to focus on strict optimization [**kirkpatrick1983optimization**].

2.3 Evolution of Timetable Management Systems

While academic research has heavily focused on the mathematical generation of schedules, the practical software engineering aspects of timetable management—how the data is ingested, visualized, and distributed—have evolved significantly alongside broader trends in software development.

2.3.1 Legacy Systems: Spreadsheets and Desktop Applications

Historically, timetables were drafted manually on large whiteboards or physical grids. With the advent of personal computing, spreadsheet software like Microsoft Excel became the defacto standard for academic coordinators. Excel offers a highly flexible,

human-readable grid interface. However, as noted by numerous educational technologists, spreadsheets are fundamentally static. They lack the relational data models necessary to automatically enforce referential integrity or detect complex logical clashes without the use of fragile, custom-built macros (VBA scripts) [**panko1998what**]. Furthermore, distributing an Excel file means distributing a static snapshot in time, leading to severe communication breakdowns when schedules inevitably change mid-semester.

In response to the limitations of spreadsheets, institutional desktop applications were developed in the late 1990s and early 2000s using technologies like Java Swing or .NET WinForms. These systems introduced relational databases (like MySQL or Oracle) to store scheduling data, offering significantly better integrity than flat files. However, being desktop-bound, they still suffered from distribution issues. Stakeholders (students and faculty) typically could not interact directly with the system, relying instead on printed reports or static HTML exports generated by administrators.

2.3.2 Modern Web-Based Architectures

The shift towards cloud computing and modern web frameworks has revolutionized how institutions handle internal logistics. Modern systems utilize full-stack web architectures, allowing centralized data storage with ubiquitous access via web browsers.

The adoption of modern frontend frameworks like React, Angular, or Vue.js has enabled the creation of Highly Interactive Single Page Applications (SPAs). Unlike older web technologies that required full page reloads for every interaction, SPAs provide dynamic, fluid user experiences. This is particularly crucial for timetabling interfaces, where users need to rapidly switch between different views (e.g., moving from a "Room View" to a "Professor View") or apply complex filters (e.g., viewing only specific student sections) without

latency.

Furthermore, the rise of Backend-as-a-Service (BaaS) platforms, such as Firebase or Supabase, has accelerated the development of such tools. By abstracting the complexities of server maintenance, database provisioning, and authentication, developers can focus more on the domain-specific logic—such as conflict detection algorithms or Excel parsing heuristics.

2.4 Identification of Gaps in Existing Solutions

Despite the proliferation of generic scheduling tools and complex academic research on constraint solving, a noticeable gap remains for many mid-to-large scale academic institutions, particularly in developing nations.

1. **The Ingestion Bottleneck:** Commercial systems often require data to be entered manually through their proprietary interfaces or imported using highly rigid CSV templates. Institutions that rely on complex, idiosyncratic Excel formats (with merged cells, color-coding, and embedded textual notes) find it extremely difficult to migrate their existing workflows into these rigid systems. There is a critical need for heuristic parsers capable of understanding "human-formatted" spreadsheets.

2. **Lack of Integrated Occupancy Tracking:** Many timetabling systems focus solely on the student/faculty schedule output. They lack dedicated tools for campus administrators to view real-time infrastructure utilization. Features like a "Free Room Finder" or a macroscopic "Master Occupancy Grid" are rarely integrated seamlessly with the core scheduling engine.

3. **Disconnect Between Generation and Management:** Academic tools that generate perfect schedules using Genetic Algorithms often feature poor, academic user interfaces that are unusable by administrative staff. Conversely, polished commercial management systems often treat the schedule as a static entity to be displayed, rather than a dynamic puzzle to be solved.

The literature thus establishes a clear trajectory: from purely mathematical algorithms that fail to account for human factors, to localized spreadsheet software that lacks systemic integration, to modern cloud-based systems that attempt to unify these domains.

TABLE 2.1: Comparison of Timetabling Methodologies

Feature	**Spreadsheets (Legacy)**	**Pure Algorithms**	**Modern Cloud Systems**
Conflict Detection	Manual & Error-Prone	Fully Automated	Real-time Automated
Data Accessibility	Offline / Localized	Varies (often CLI based)	Cloud-synchronized (Web/Mobile)
Flexibility	High (free-form data entry)	Low (strict mathematical constraints)	High (heuristics + constraints)
Integration	None	Low	High (APIs, Database links)
User Interface	Basic Grid	Academic / Non-existent	Role-based Dashboards

The system developed in this thesis builds upon this foundation, adopting the "Modern Cloud System" paradigm while specifically integrating a heuristic parser to bridge the gap with legacy spreadsheet data, laying a clean architectural foundation for future automated constraint-solving integration.

Chapter 3

Technologies and Frameworks

The IIITA Timetabling System utilizes a modern, browser-based stack selected to prioritize high-performance data processing and strict referential integrity. This chapter outlines the technical foundation, focusing on tools that facilitate complex client-side computation and the normalization of legacy institutional data. [cite: 163]

TABLE 3.1: Summary of the Technology Stack

Layer	Technology	Primary Function in TTGen
Frontend Framework	React 19	Rendering dense, interactive 5-day time grids. [cite: 279]
Programming Language	TypeScript	Enforcing strict data contracts for slots and genes. [cite: 279]
Database	PostgreSQL	Maintaining integrity across rooms, faculty, and subjects. [cite: 279]
Backend / BaaS	Supabase	API gateway and secure database interfacing. [cite: 350]
Excel Parsing	SheetJS (xlsx)	Heuristic extraction of semantic meaning from legacy files. [cite: 370]

3.1 Frontend Architecture: React 19 and TypeScript

The UI is designed to manage high-density data interactions without performance degradation. To achieve this, the system leverages React's virtual DOM and TypeScript's static typing. [cite: 295, 307]

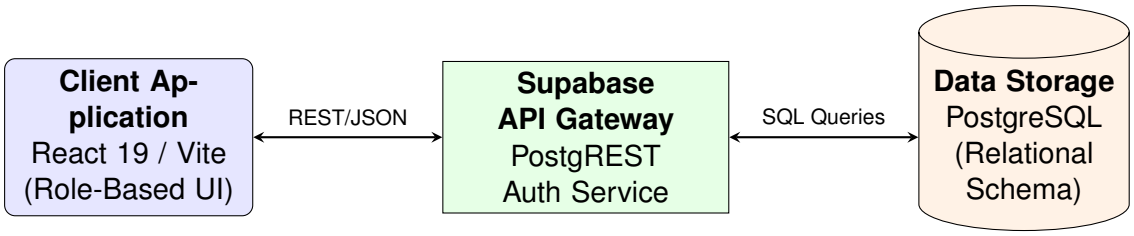

FIGURE 3.1: High-Level System Architecture and Component Interaction
[cite: 288]

3.1.1 Performance and Data Reliability

The system employs the `useMemo` hook to memoize expensive computations—such as $O(N^2)$ clash detection—ensuring these logic-heavy tasks only execute when the underlying timetable data changes. [cite: 304] TypeScript is used to define rigid data interfaces, such as the `ParsedSlot`, ensuring the heuristic Excel parser outputs structured, predictable data. [cite: 309, 310]

```
// Example TypeScript Interface for Solver Data Contracts

interface ParsedSlot {

    id: string;

    day: string;

    startTime: string;

    subjectCode: string;

    type: 'Lecture' | 'Tutorial' | 'Practical';

    roomName: string;

    section: string;

} [cite: 312]
```

3.2 Build and Styling: Vite and Tailwind CSS

The development workflow and visual layer are optimized for rapid iteration and dynamic feedback. [cite: 333]

- **Vite:** Leverages native ES Modules for near-instantaneous hot-module replacement (HMR), critical during the refinement of grid-generation algorithms. [cite: 332]
- **Tailwind CSS:** A utility-first framework that allows the system to dynamically apply visual alerts, such as red ring borders for clashing sessions, directly within the React components. [cite: 338, 343]

3.3 Backend and Data Ingestion: Supabase

The system replaces static files with a centralized PostgreSQL database hosted on Supabase, ensuring referential integrity through foreign key constraints. [cite: 350, 354]

- **Heuristic Parsing:** The `xlsx` (SheetJS) library is used to read raw institutional spreadsheets. [cite: 370] These are processed by a custom engine to extract semantic meaning, such as L-T-P credit structures and complex faculty assignments. [cite: 551, 552]
- **Authentication:** Secure access for administrators is managed natively by Supabase Auth, with state handled via a global `AuthContext` provider. [cite: 365, 366]

3.4 Solver Engine: Pure TypeScript Architecture

The core contribution of this system is its pure-TypeScript solver engine, which runs directly in the browser. [cite: 163, 391] This eliminates the need for server-side optimization and allows for real-time schedule generation. The solver is organized into discrete submodules: [cite: 393]

- `dataPrep.ts`: Expands high-level credits into individual sessions and applies the 2+1 lecture format. [cite: 401]
- `constraints.ts`: Evaluates solutions against ten hard and three soft constraints. [cite: 403, 404]
- `mutations.ts`: Implements five mutation operators, including day reassignment and Gaussian time shifts. [cite: 406, 407]
- `solver.ts`: Manages the (1+1)-ES loop, elitist selection, and stagnation restarts. [cite: 411, 412]
- `localSearch.ts`: Provides the LNS auto-fix routine for the interactive editor. [cite: 414, 415]

3.5 External Integrations: Reporting and Export

To support administrative workflows, the system integrates with external reporting tools: [cite: 367]

- **Google Sheets API:** Exports formatted schedules with frozen headers and color-coded session types directly to Google Drive. [cite: 377, 381]

- **PDF Generation:** Captures high-fidelity snapshots of the timetable grid using `html-to-image` and `jsPDF` for immediate offline use. [cite: 384, 386]

Chapter 4

System Architecture and Design

In this chapter, we walk through the structural design of the IIITA Timetabling System. We cover the overall application topology, the relational database schema, how we ingest data from Excel files, the solver module architecture, and the routing logic that ties everything together.

4.1 Application Topology

Our system follows a classic three-tier architecture adapted for the modern web using a Backend-as-a-Service (BaaS) model.

- **Client Tier (React SPA):** The frontend is a Single Page Application (SPA) responsible for all user interactions, data formatting, client-side clash detection, and PDF generation. It communicates asynchronously with the backend via secure REST API calls over HTTPS.
- **Backend Tier (Supabase):** Acting as the serverless backend, Supabase provides an auto-generated API gateway, handles user authentication, and interfaces directly with the underlying database.

- **Data Tier (PostgreSQL):** The relational database stores all normalized institutional data, including users, entities (rooms, subjects, professors), and the core timetable slots.

4.2 Database Schema Design

A well-structured relational schema is the backbone of any scheduling system. Our database is heavily normalised to prevent data duplication and enforce referential integrity. It contains eight tables in total—six core entities and two solver-support mapping tables. Figure 4.1 shows the full entity-relationship diagram.

4.2.1 Core Entities

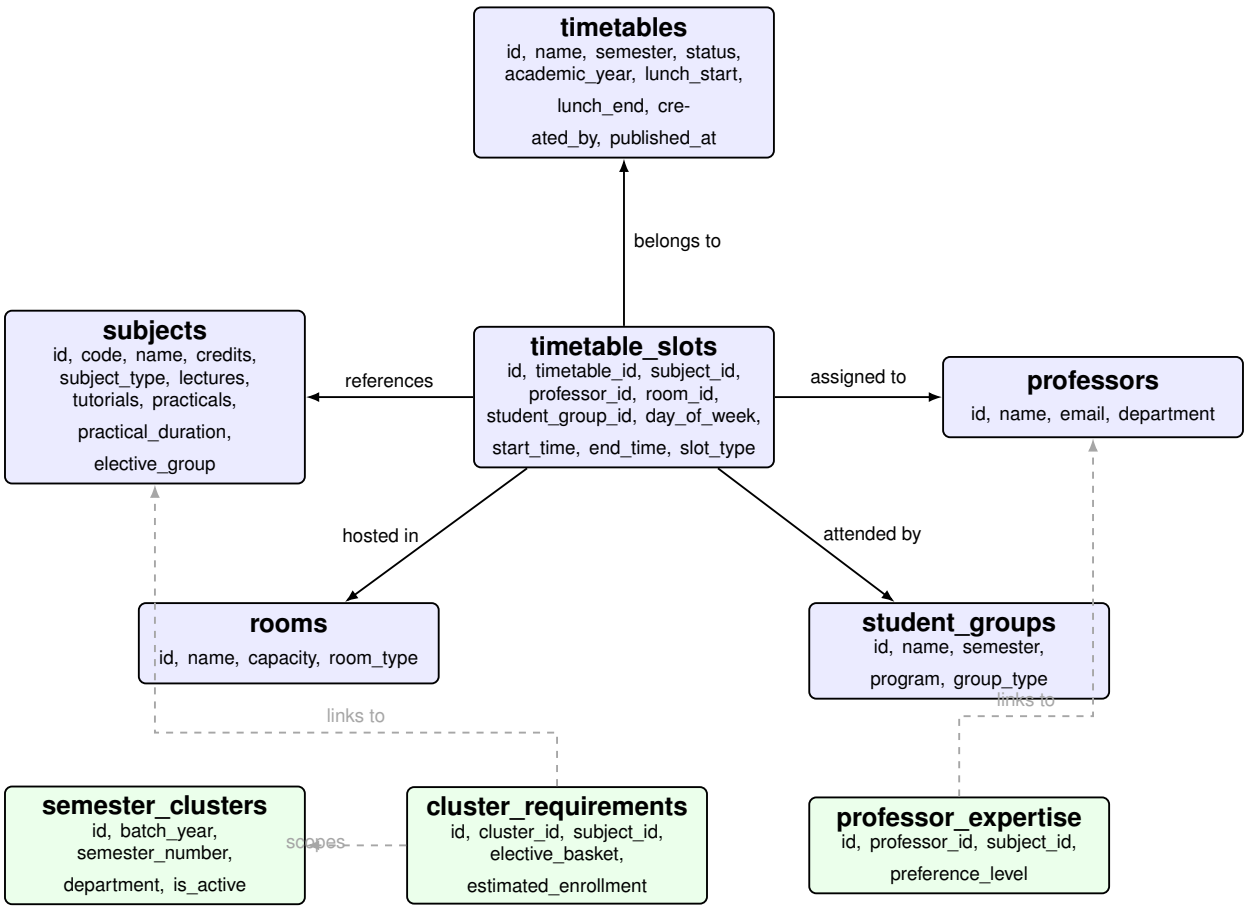

FIGURE 4.1: Entity-Relationship Diagram of the Complete Database Schema

- **timetables:** The parent container for a schedule iteration (e.g., "IT Sem-5 (2023)"). Key attributes include `name`, `semester`, `academic_year` (e.g., "2025-26"), `status` (draft or published), `lunch_start`/`lunch_end` (defaults: 13:00–14:30), `created_by` (the admin user), and `published_at`.
- **subjects:** Stores all offered courses. Attributes include the unique `code`, `name`, total `credits`, `subject_type` (Core, Elective, or Minor), L-T-P breakdown (`lectures`, `tutorials`, `practicals`), `practical_duration` (in minutes), and `elective_group` (e.g., "HSMC", "Basket 1").
- **professors:** Faculty members, uniquely identified by `email`, with `name` and `department`.
- **rooms:** Physical campus rooms. Attributes include the unique `name` following the `CC-{building}-{number}` convention (e.g., "CC-3-5106", "CC-3-5207"), seating `capacity` (default 60), and `room_type` (Lecture or Lab—derived from whether the room hosts only Practical sessions).
- **student_groups:** A student cohort defined by `name` (e.g., "Sec A", "Sec B", "WMC" for whole-batch, or "IT-BI" for the IT-BI programme), `semester`, `program` ("B.Tech"), and `group_type` ("Core"). The unique constraint is on `(name, semester)`.

4.2.2 The Central Hub: `timetable_slots`

The `timetable_slots` table is the heart of the schema—a massive junction table that links a specific timetable to a subject, professor, room, and student group at a particular point in time. Table 4.1 lists every column.

TABLE 4.1: Schema Specification for `timetable_slots`

Column Name	Data Type	Description & Constraints
`id`	`UUID`	Primary Key, auto-generated.
`timetable_id`	`UUID`	Foreign Key referencing `timetables`.
`subject_id`	`UUID`	Foreign Key referencing `subjects`.
`professor_id`	`UUID`	Foreign Key referencing `professors`.
`room_id`	`UUID`	Foreign Key referencing `rooms`.
`student_group_id`	`UUID`	Foreign Key referencing `student_groups`.
`day_of_week`	`Integer`	$1 =$ Monday, $\ldots, 5 =$ Friday.
`start_time`	`VarChar`	24-hour format (e.g., "09:00").
`end_time`	`VarChar`	24-hour format (e.g., "11:00").
`slot_type`	`VarChar`	Enum: 'Lecture', 'Tutorial', 'Practical'.

4.2.3 Generator and Solver Support Entities

To support automated timetable generation, the schema includes three mapping tables (shown in green in Figure 4.1). These are used by the Intelligent Generator and the solver engine:

- **`semester_clusters`:** Defines active academic clusters (e.g., Batch 2023, Semester 3, Department "IT"). Attributes include `batch_year`, `semester_number`, `department`, and `is_active`. The generator UI filters for active clusters and uses them to scope the scheduling problem.
- **`cluster_requirements`:** A many-to-many join linking a cluster to the subjects that must be scheduled. Includes `elective_basket` (the basket name for elective grouping) and `estimated_enrollment` (used by the solver's room utilisation soft constraint).
- **`professor_expertise`:** Links professors to the subjects they are qualified to teach, along with a `preference_level`. The auto-fill feature in the Intelligent Generator

uses this to intelligently suggest professor assignments.

4.2.4 Solver Data Model: Class Diagram

Figure 4.2 presents a UML class diagram of the solver's core data model, showing the relationships between the key TypeScript interfaces that drive the scheduling engine.

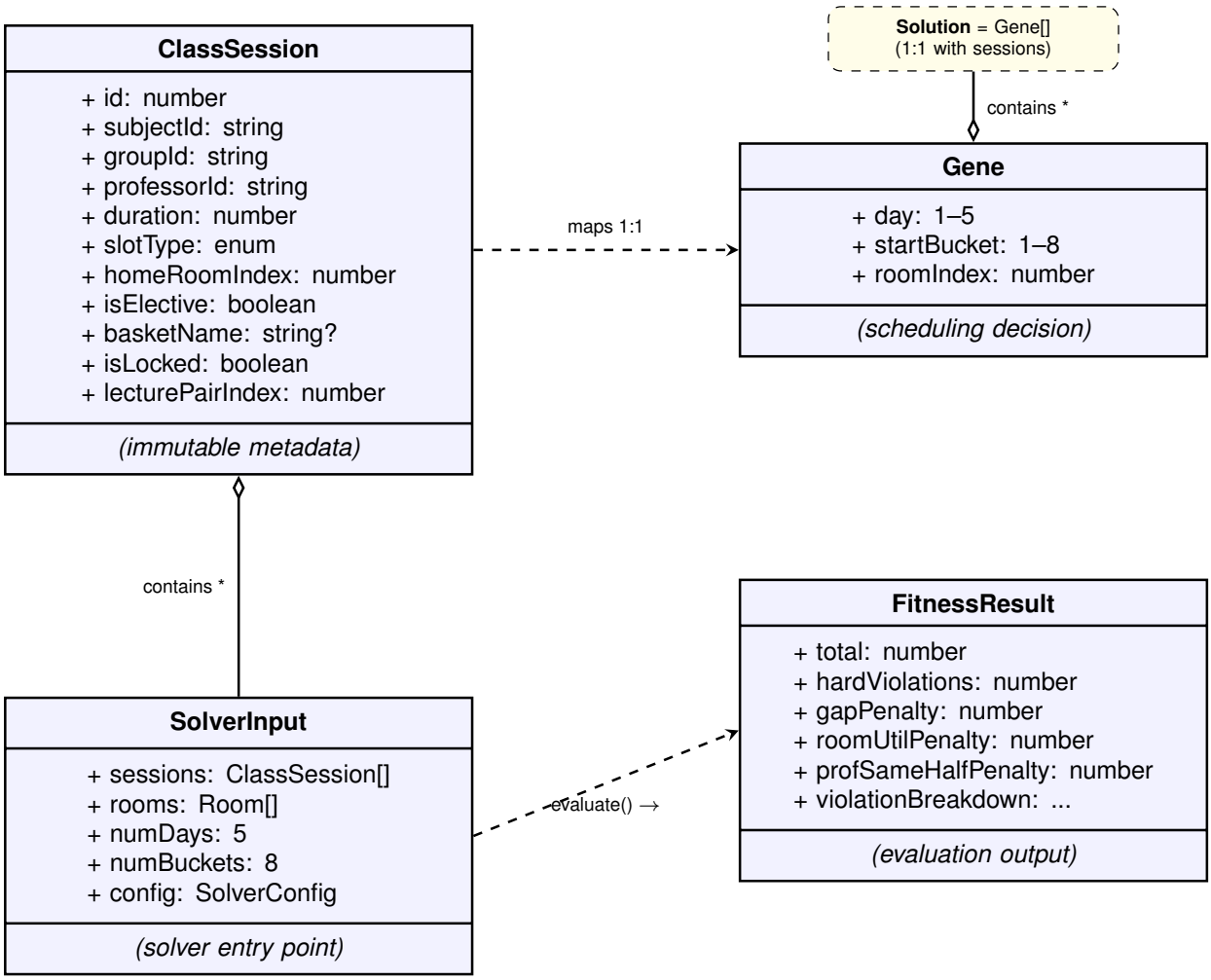

FIGURE 4.2: UML Class Diagram of the Solver Data Model

4.3 The Data Ingestion Pipeline

A major architectural feature of the system is its ability to ingest unstructured data. The transition from an administrator's raw Excel file to normalized database records follows a strict pipeline managed by the `TimetableImporter` module:

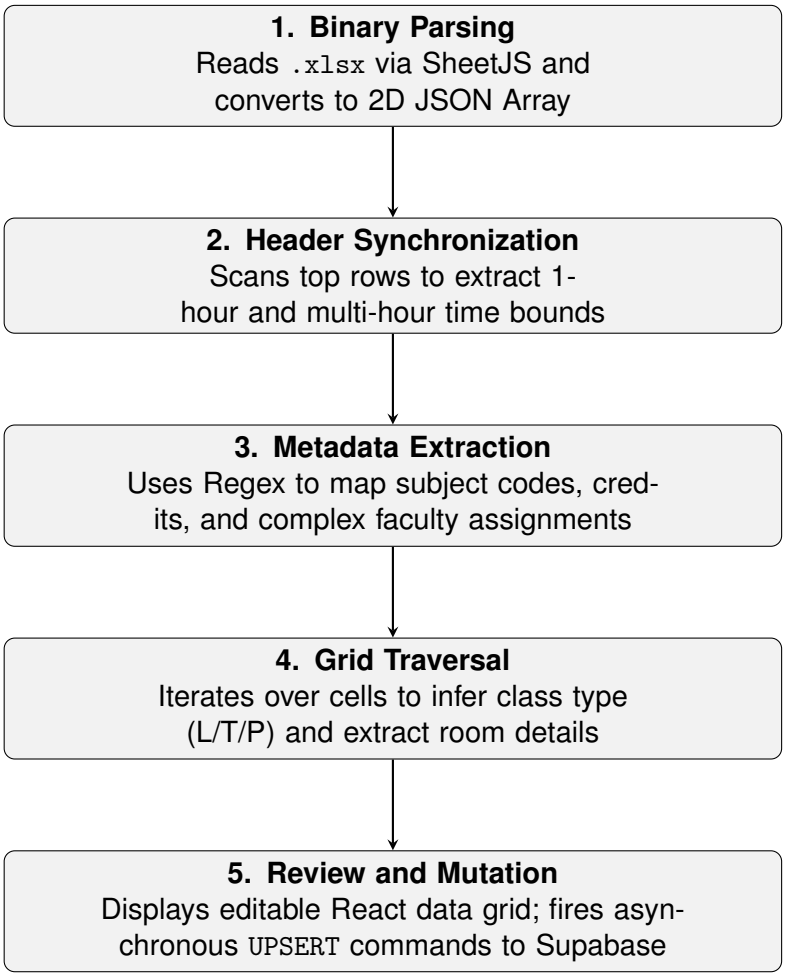

FIGURE 4.3: Data Ingestion Pipeline for Excel Timetables

1. **Binary Parsing:** The user uploads an '.xlsx' file. The 'xlsx' library reads the binary stream and converts the active spreadsheet into a two-dimensional JSON array.

2. **Header Synchronization:** The heuristic engine scans Row 2 to identify time boundary headers. It converts both colon and dot formats (e.g., "9:00-10:00" or "9.00-10.00") into normalised 24-hour strings and detects multi-hour merged-cell blocks for practical sessions.

3. **Metadata Extraction:** The engine locates the "Course Code / Faculties" section of the spreadsheet. It applies Regular Expressions to map subject codes to full names and extracts the L-T-P-S credit structure (e.g., "3-1-0-0"). It also parses complex faculty assignment strings—for instance, identifying that "Dr. Sharma(A & B), Dr. Gupta(C & D)" means Dr. Sharma teaches Sections A and B while Dr. Gupta teaches Sections C and D.

4. **Grid Traversal and Extraction:** The engine iterates over the main timetable grid. For each cell, it:

   - Identifies the current day from the leftmost column.
   - Identifies the time bounds from the column header.
   - Extracts the subject code, infers the class type (L/T/P) based on cell coloring or text markers, and extracts room and section assignments.

5. **Review and Mutation:** The extracted raw data is presented to the administrator in an editable React data grid. Upon confirmation, the application fires a sequence of asynchronous `UPSERT` commands to Supabase, ensuring that all necessary entities (professors, rooms, subjects) exist before performing a bulk `INSERT` into the `timetable_slots` table.

4.4 View Routing and State Management

The application does not utilize a traditional URL-based router (like React Router). Instead, it employs state-based view switching managed by the root ‘App.tsx‘ component.

A central state variable, ‘currentView‘, dictates which heavy component is mounted into the main viewing area. The available views are:

- `import`: Mounts the `TimetableImporter`.
- `generator`: Mounts the multi-step `GeneratorView` wizard for automated timetable creation.
- `editor`: Mounts the interactive `EditorView` for drag-and-drop timetable refinement with LNS auto-fix.
- `student`: Mounts the `TimetableViewer` (All Students administrative view).
- `prof`: Mounts the `ProfessorTimetableViewer`.
- `room`: Mounts the `RoomTimetableViewer`.
- `master-room`: Mounts the campus-wide `MasterRoomViewer`.
- `free-room`: Mounts the `FreeRoomViewer`.
- `report-card`: Mounts the `ReportCardView`.

This architecture ensures that only the data required for the active view is fetched from the database, minimizing unnecessary network traffic. Custom React hooks, notably `useGeneratorData` and `useEditorData`, manage the fetching and caching of complex relational data required by these components, abstracting the Supabase query syntax away from the presentation layer.

Chapter 5

Implementation and Results

This is where everything comes together. In this chapter, we walk through the key algorithms and interfaces we built for the IIITA Timetabling System—from the dynamic grid rendering and real-time conflict detection, through to the (1+1)-ES solver, the interactive drag-and-drop editor, and the multi-step Intelligent Generator. Along the way, we explain the design decisions we made and why.

5.1 Dynamic Time Grid Generation

A significant challenge in displaying timetables is the variability of time slots. Hardcoding a static grid (e.g., exactly ten 1-hour slots from 9:00 AM to 7:00 PM) results in wasted visual space when classes end early, and fails to accommodate non-standard time ranges (such as a 1.5-hour class).

To solve this, all viewer components in the system implement a **Dynamic Time Column Generator algorithm**. When a timetable is selected, the system fetches all associated `timetable_slots`. The algorithm then executes the following steps:

1. Extracts all unique `start_time` and `end_time` strings from the raw slot data.

2. Injects hardcoded institutional break times: `10:50` to `11:00` (Morning Break) and `13:00` to `14:30` (Lunch Break).
3. Sorts all extracted timestamps chronologically to create an array of discrete time boundaries.
4. Iterates through adjacent pairs of boundaries to generate standard columns.
5. Filters out any generated columns that fall entirely within the lunch or break periods, or that contain absolutely zero scheduled classes.

The result is a highly responsive grid that only displays columns for active periods, maximizing screen real estate and improving readability.

5.2 Multi-Dimensional Clash Detection Algorithm

One of the most critical administrative features is the automated detection of scheduling conflicts. The 'TimetableViewer' component implements a robust $O(N^2)$ algorithm that compares every slot against every other slot within the selected timetable to detect overlaps.

An overlap is mathematically defined as occurring when:

$$(SlotA.start < SlotB.end) \wedge (SlotA.end > SlotB.start) \tag{5.1}$$

When an overlap is detected, the algorithm categorizes the clash into one of four distinct types as summarized in Table 5.1.

TABLE 5.1: Types of Scheduling Clashes Detected

Clash Type	Logical Condition and Description
Room Clash	$SlotA.room == SlotB.room$ Two different classes are assigned to the exact same physical space simultaneously.
Professor Clash	$SlotA.professor == SlotB.professor$ A single faculty member is booked to teach two different classes at the same time.
Section Clash	$SlotA.section == SlotB.section \land SlotA.subject \neq SlotB.subject$ A specific student cohort is expected to attend two distinct subjects simultaneously.
WMC-Section	Specialized institutional rule where generic weekend/WMC sessions overlap with standard section sessions.

These clashes are immediately surfaced to the administrator in a dedicated visual panel, complete with distinct iconography (e.g., a Map Pin for Room clashes, a User icon for Professor clashes), allowing for rapid identification and remediation.

5.3 Inverted Logic: The Free Room Finder

Traditional scheduling systems focus on when rooms are occupied. However, students and staff frequently need to know when rooms are *available*. The ‘FreeRoomViewer‘ component achieves this through an inverted logic algorithm.

Unlike the dynamic grid, the Free Room Finder establishes a hardcoded temporal matrix (e.g., 8:50 AM to 6:30 PM in discrete blocks).

The algorithm executes as follows:

1. The system first queries the database for **all** existing rooms.

2. It then queries the `timetable_slots` to retrieve all currently active sessions.

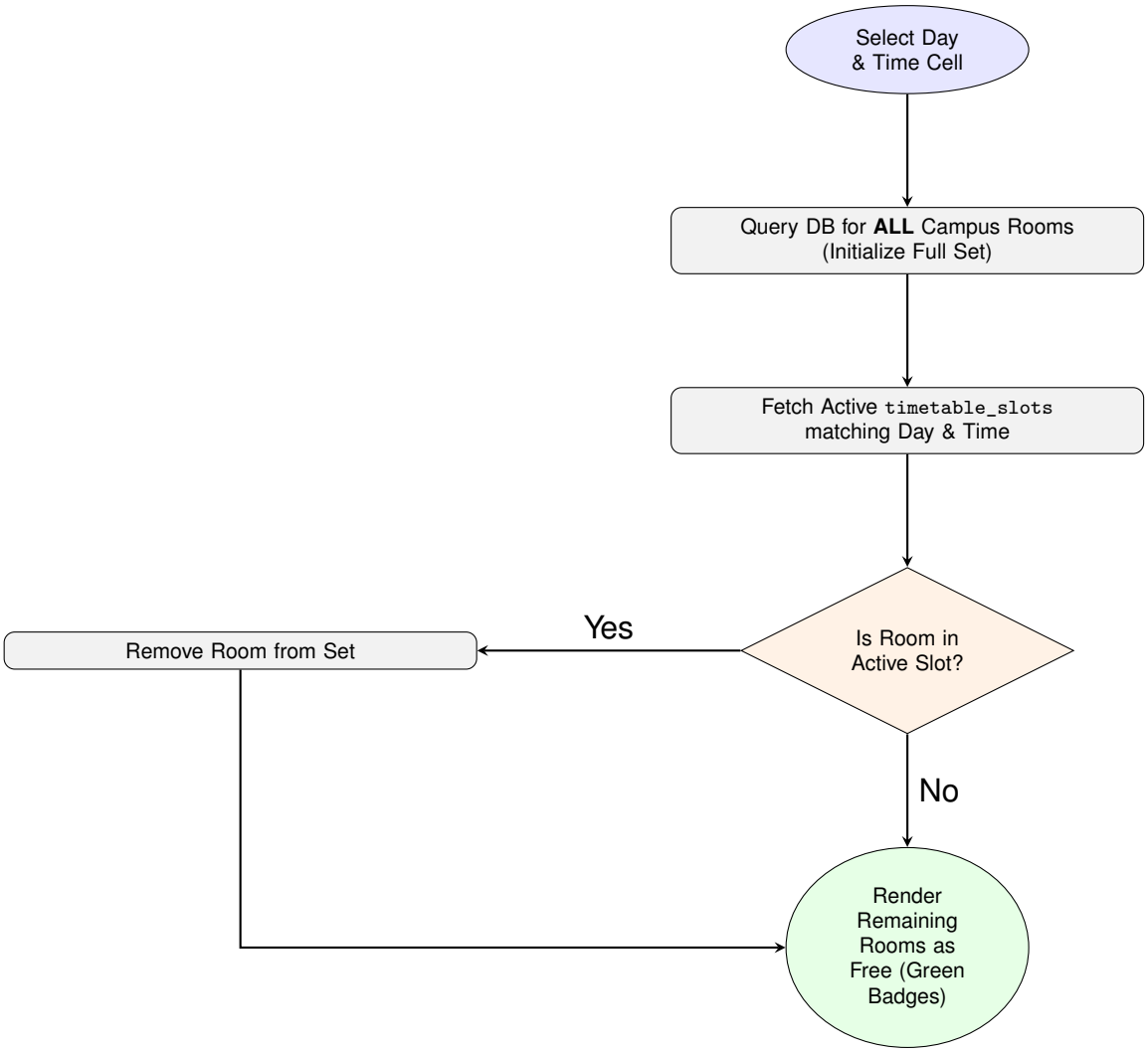

FIGURE 5.1: Inverted Logic Flowchart for the Free Room Finder Algorithm

3. For every cell in the temporal matrix (Day $\times$ Time), it instantiates the complete set of all campus rooms.

4. It iterates through the active sessions. If a session falls within the current cell's Day and Time, that session's `room_id` is removed from the set of available rooms.

The remaining rooms in the set are then rendered in the UI as green badges, instantly showing users where they can find empty spaces across the campus.

5.4 React Component Hierarchy

To maintain a scalable and manageable codebase, the frontend relies on a strict component tree hierarchy. Figure 5.2 illustrates the relationship between the global layout and the localized, role-based timetable viewers.

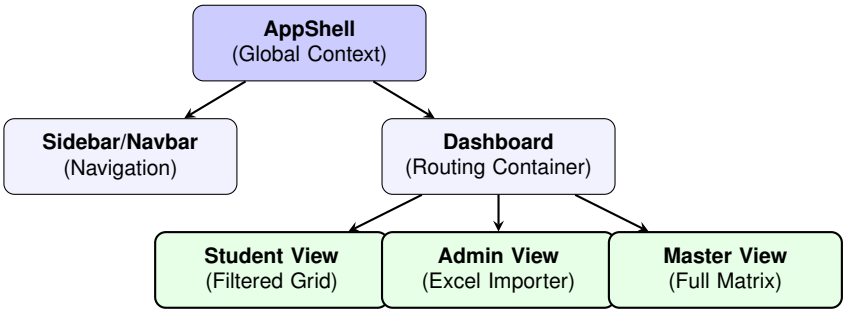

FIGURE 5.2: Hierarchical Structure of the Core React Components

5.5 Role-Based Viewer Implementations

The system provides several specialized views tailored to different stakeholders, summarized in Table 5.2.

TABLE 5.2: Role-Based Component Mapping and Data Scope

Stakeholder Role	Component View	Primary Filter Parameter
Student	`TimetableViewer`	Semester & Specific Section
Professor	`ProfessorTimetableViewer`	Faculty Email/ID
Department Admin	`RoomTimetableViewer`	Specific Room Name
Campus Admin	`MasterRoomViewer`	Building prefix (e.g., “CC-3”, “LT”)
Facility Staff	`FreeRoomViewer`	Temporal Matrix (Empty Rooms)

5.5.1 Student Cohort View

Given the complexity of modern curricula featuring numerous electives and minor tracks, a master timetable can be overwhelming. The `TimetableViewer` allows users to filter the master schedule by selecting a specific semester and section.

Crucially, it implements a **Smart Merging algorithm**. Often, core lectures are held for "All Sections" simultaneously, while practical laboratory sessions are split by specific section (e.g., "Sec A"). The merging algorithm intelligently filters the master data, pulling in slots explicitly assigned to the selected section *and* slots assigned to the generic "All" section (WMC), combining them into a single, cohesive cohort schedule.

5.5.2 Master Occupancy Viewer

For campus administrators, the ‘MasterRoomViewer‘ provides a macro-level perspective. It displays a dense grid where rows represent physical rooms rather than student groups. Because an institution may have hundreds of rooms, this view implements a building-level filter. The system extracts the building prefix from the room name (e.g., identifying “CC-3” from “CC-3-5106”) and allows the administrator to filter the grid, rendering a manageable overview of specific campus zones.

5.6 Export Mechanisms and Results

To bridge the gap between the digital system and offline requirements, the platform implements robust export mechanisms.

5.6.1 Client-Side PDF Generation

Users can generate PDFs of their current view. This relies on the ‘html-to-image‘ library to capture the DOM state as a PNG, which is then embedded into a ‘jsPDF‘ document. A significant technical hurdle overcome during implementation was the handling of sticky CSS headers; the capture logic temporarily disables ‘position: sticky‘ properties on the DOM elements right before capture to ensure the entire scrollable area is rendered to the image buffer accurately.

5.6.2 Google Sheets API Export

The ‘RoomTimetableViewer‘ and ‘FreeRoomViewer‘ support bulk export directly to Google Sheets. Using the Google Identity Services client, the React application constructs a multi-layered batch update request. This request not only writes the raw string data into the cells but explicitly formats the workbook—creating separate worksheets for different rooms, freezing header rows, applying pink and gray background colors to headers, and drawing solid borders—resulting in a highly professional, ready-to-share document directly in the administrator’s Google Drive.

5.6.3 User Interface Evaluation

The resulting application is a highly responsive, modern interface. By utilizing Tailwind CSS, the application maintains a clean, minimalist aesthetic with clear color coding (e.g.,

orange for practicals, green for tutorials). The elimination of page reloads between views, combined with the instantaneous clash detection, represents a monumental upgrade in usability and efficiency compared to traditional spreadsheet workflows.

5.7 The (1+1)-ES Timetable Solver

The automated timetable generator is the centrepiece of this project. We use a (1+1) Evolution Strategy (ES) [**rechenberg1973evolution**, **schwefel1995evolution**]—a simple but effective meta-heuristic—to search for conflict-free schedules. In the subsections that follow, we formalise the problem representation, describe how we score candidates, and explain the mutation and adaptation mechanisms that drive the search.

5.7.1 Chromosome Representation

The solver models a complete timetable as a **Solution**, which is an ordered array of **Gene** objects. Each Gene g_i encodes the scheduling decision for the i-th class session and is defined as:

$$g_i = (\text{day}_i, \text{startBucket}_i, \text{roomIndex}_i) \tag{5.2}$$

where:

- $\text{day}_i \in \{1, 2, 3, 4, 5\}$ corresponds to Monday through Friday.
- $\text{startBucket}_i \in \{1, 2, \ldots, 8\}$ is the starting time slot (each slot represents one hour, from 08:50 to 18:30).
- roomIndex_i is the index into the available room array.

Each class session is represented by a `ClassSession` object containing immutable metadata: the subject identifier, student group, assigned professor, session duration, slot type (Lecture, Tutorial, or Practical), home room index, elective basket membership, and a locked flag indicating whether the session belongs to a previously published timetable.

5.7.2 Session Expansion: The 2+1 Lecture Format

Before solving, the `dataPrep.ts` module transforms high-level subject configurations (L-T-P counts) into individual `ClassSession` entries. A key institutional rule is the **2+1 lecture format**: for core subjects with $L \geq 2$ lectures per week, the system creates:

- Exactly **one** double-lecture block of duration 2 hours (`lecturePairIndex = 0`).
- The remaining $L - 2$ lectures as individual 1-hour sessions (`lecturePairIndex = -1`).

This is enforced as a hard constraint: the double-lecture and all single-lectures for the same subject and group must be placed on *different* days.

5.7.3 Fitness Function

The quality of a candidate solution S is evaluated by a composite fitness function:

$$f(S) = H(S) \cdot W_{\text{hard}} + G(S) \cdot W_{\text{gap}} + R(S) \cdot W_{\text{room}} + P(S) \cdot W_{\text{prof}} \tag{5.3}$$

where:

- $H(S)$ is the total number of hard constraint violations.
- $G(S)$ is the student schedule gap penalty (soft).

- $R(S)$ is the room utilization penalty (soft).
- $P(S)$ is the professor same-half penalty (soft).
- $W_{\text{hard}} = 1000$, $W_{\text{gap}} = 1.0$, $W_{\text{room}} = 0.8$, $W_{\text{prof}} = 5.0$ are configurable weights.

The large hard penalty weight ($W_{\text{hard}} = 1000$) ensures that the solver prioritizes eliminating all hard violations before optimizing soft constraints. A feasible solution has $H(S) = 0$.

5.7.4 Hard Constraints

The system enforces ten hard constraints, summarized in Table 5.3.

TABLE 5.3: Hard Constraints Enforced by the Solver

#	Constraint Name	Description
1	Time Boundary	A session must not extend beyond slot 8 (18:30).
2	Break/Lunch Crossing	A multi-slot session must not span across a break boundary (slots 2→3 or 4→5).
3	Room Non-Overlap	At most one session may occupy a given room at any (day, slot).
4	Professor Non-Overlap	A professor can teach at most one session at any given slot. No elective exemption.
5	Group Non-Overlap	A student group can attend at most one session per slot. Elective–elective pairs and same-basket pairs are exempt.
6	Elective Basket Sync	Synced baskets (HSMC, MDM) must share the same (day, slot). No two different baskets may share a slot (global anti-clash).
7	Lab Room	Practical sessions must be assigned to rooms with type “Lab”.
8	WMC-Section Overlap	A whole-batch (WMC) session cannot occupy the same slot as any section-level session.
9	Home Room	Core lectures and tutorials must be placed in their assigned home room.
10	2+1 Lecture Format	The double-lecture block and single-lectures for the same subject+group must be on different days.

5.7.5 Soft Constraints

Three soft constraints provide schedule quality optimization beyond feasibility:

1. **Student Gap Penalty $G(S)$:** For each student group on each day, the penalty counts empty slots between their first and last scheduled class. Formally, if a group occupies a set of slots $\mathcal{S}_d$ on day d, the gap is $(\max(\mathcal{S}_d) - \min(\mathcal{S}_d) + 1) - |\mathcal{S}_d|$.

2. **Room Utilization Penalty $R(S)$:** For practical and elective sessions, the solver computes the utilization ratio $u = \text{studentCount}/\text{roomCapacity}$. If $u < 0.60$ (under-utilization) or $u > 1.20$ (over-utilization), a proportional penalty is applied. The result is scaled to an integer range by multiplying by 100.

3. **Professor Same-Half Penalty $P(S)$:** For each professor on each day, a penalty of 1 is incurred if they teach in both the morning half (slots 1–4) and the afternoon half (slots 5–8). This encourages compact teaching schedules.

5.7.6 Mutation Operators

The (1+1)-ES generates offspring by applying a small number of mutations to the parent solution. Each generation, the solver randomly selects 3–8% of mutable (non-locked) sessions and applies one of five mutation operators:

1. **Day Reassignment:** Assigns a uniformly random new day $d \in \{1, \ldots, 5\}$.

2. **Gaussian Time Shift:** Perturbs the start slot by $\Delta = \lfloor \mathcal{N}(0, \sigma^2) \rceil$, clamped to valid bounds. If the new position crosses a break boundary, the slot is snapped to the nearest valid start using a precomputed cache.

3. **Type-Aware Room Reassignment:** Assigns a random room respecting the session type: Lab rooms for practicals, lecture rooms for electives, and the fixed home room for core lectures/tutorials.

4. **Swap:** Exchanges the (day, slot, room) of two same-duration sessions. Synced-basket electives are excluded from swapping to preserve group synchronization.

5. **Full Relocate:** Assigns a completely new random (day, slot, room) triple.

For synced-basket electives (HSMC, MDM), mutations to day, time, or relocation are *propagated* to all members of the sync group, ensuring they remain synchronized.

5.7.7 σ-Adaptation: The 1/5th Success Rule

The step-size parameter σ controls the magnitude of Gaussian time perturbations. The solver adapts σ using Rechenberg's 1/5th success rule [**rechenberg1973evolution**]. Every $W = 50$ generations (the adaptation window), the success rate p_s (fraction of offspring that improved fitness) is computed:

$$\sigma \leftarrow \begin{cases} \sigma \times 1.22 & \text{if } p_s > 1/5 \quad \text{(too many successes} \Rightarrow \text{increase exploration)} \\ \sigma \times 0.82 & \text{if } p_s < 1/5 \quad \text{(too few successes} \Rightarrow \text{decrease step size)} \\ \sigma & \text{otherwise} \end{cases} \tag{5.4}$$

This adaptive mechanism balances exploration (large jumps in the search space) and exploitation (fine-tuning near a local optimum).

5.7.8 Stagnation Restart

If the best fitness does not improve for a configurable number of consecutive generations (the stagnation threshold, typically 25% of total generations), the solver performs a **restart**: the current solution is discarded and a fresh random initial solution is generated. The best-ever solution is preserved across restarts via elitist tracking.

5.7.9 Locked Sessions and Cross-Semester Awareness

When generating a timetable for a specific semester, the solver can incorporate **locked sessions** from previously published timetables of other semesters. These locked sessions are appended to the session array with their `isLocked` flag set to `true`. Their Gene positions are fixed and never mutated. However, they participate fully in constraint evaluation—particularly room overlap and professor overlap checks—ensuring that the new timetable does not conflict with existing published schedules.

5.7.10 Solver Execution Flow

The solver runs asynchronously in the browser using batched `setTimeout` calls (processing 200 generations per batch) to maintain UI responsiveness. The overall flow is:

1. Generate an initial solution using round-robin day assignment and break-aware slot placement.
2. If a seed timetable is provided, override matched sessions with their existing positions.
3. Pin locked genes from published timetables.

4. For each generation: mutate the parent, evaluate the offspring, accept if $f(\text{offspring}) \leq f(\text{parent})$ (elitist selection).
5. Every W generations, adapt σ via the 1/5th rule.
6. If stagnation is detected, restart with a fresh random solution.
7. Report progress (current fitness, σ, success rate, violation breakdown) to the UI at configurable intervals.
8. Terminate on reaching the maximum generation count, achieving zero hard violations, or user cancellation.

Figure 5.3 visualizes this execution flow as a UML activity diagram.

5.8 The Interactive Timetable Editor

The `EditorView` component is where human judgment meets algorithmic output. It gives administrators an interactive, drag-and-drop interface for refining timetables—whether those timetables were generated by the solver or imported from Excel.

5.8.1 Drag-and-Drop Slot Rearrangement

The editor renders a dense 5-day $\times$ N-slot grid, where each scheduled session appears as a draggable card. Administrators can drag any session card from one cell to another to reassign its day and start time. The system automatically recalculates the end time based on the session's duration and validates that the new position does not fall in a lunch or break slot. All drag operations push the previous state onto a 30-level undo stack.

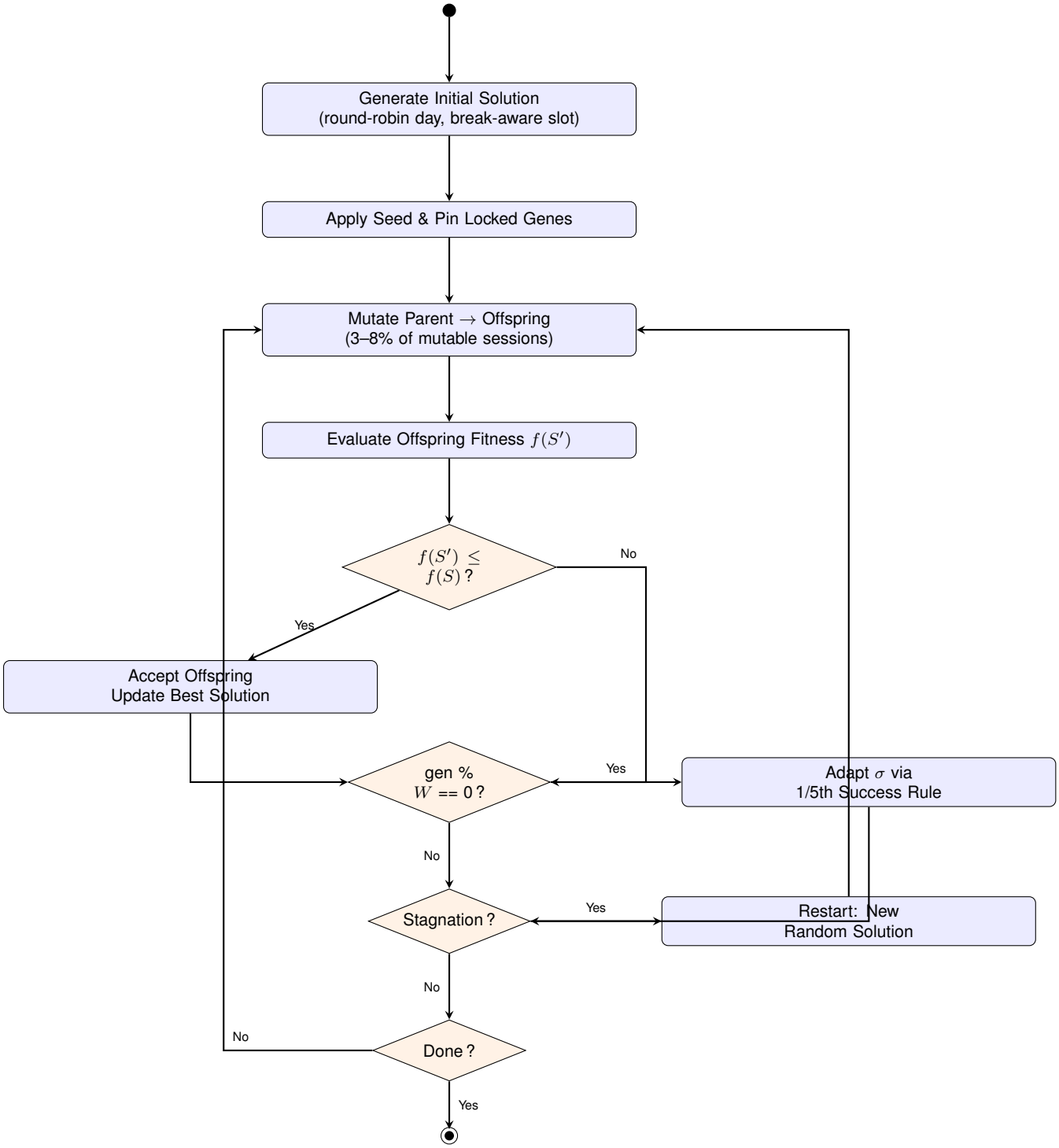

FIGURE 5.3: Activity Diagram: (1+1)-ES Solver Execution Flow

5.8.2 Inline Editing

Each session card exposes a hover-activated edit panel that allows inline modification of:

- **Room Assignment:** A cascading two-step selector where the administrator first selects the room type (Lecture or Lab), then picks from filtered rooms of that type.
- **Professor Assignment:** A dropdown populated from the full professor list.

5.8.3 Real-Time Conflict Visualization

The editor implements a client-side clash detection algorithm that runs on every state change. Sessions involved in any conflict are highlighted with a red background and ring border. The algorithm checks the same constraints as the solver: room overlaps, professor overlaps, section overlaps (with elective exemptions), WMC-section conflicts, cross-basket elective clashes, and the 2+1 same-day lecture rule.

5.8.4 Feasibility Check

A dedicated "Check Feasibility" button constructs a full `SolverInput` from the current editor state, converts the slot positions into a `Solution` (Gene array), appends locked sessions from all other published timetables, and invokes the solver's `evaluate()` function. The result displays the total fitness score and the per-constraint violation breakdown, giving the administrator a precise understanding of remaining issues.

5.8.5 LNS Auto-Fix

When conflicts exist, the administrator can invoke the **Large Neighbourhood Search (LNS)** auto-fix [**shaw1998using**]. This routine:

1. Identifies all session indices involved in any hard constraint violation (the “clashing set”).
2. Iterates for up to 1500 attempts. In each attempt, a random session from the clashing set is selected and a random mutation (relocate, time shift, or room change) is applied.
3. If the mutation reduces the total fitness, the improved solution is accepted and the clashing set is refreshed.
4. The process terminates early if all hard violations reach zero.

The LNS targets only the problematic sessions rather than reoptimizing the entire schedule, making it significantly faster than a full solver run while preserving the administrator’s manual adjustments to non-conflicting sessions. Figure 5.4 illustrates this destroy-and-repair process.

5.8.6 Master View

The editor includes a “Master View” toggle that fetches and overlays all slots from other published timetables as read-only, semi-transparent cards in the grid. This provides cross-semester situational awareness, allowing the administrator to visually verify that no room or professor conflicts exist between the current draft and previously published schedules.

5.9 The Intelligent Generator

The `GeneratorView` component is the front door to the solver. It walks administrators through a guided, step-by-step configuration process that collects all the information the solver needs before launching the optimisation.

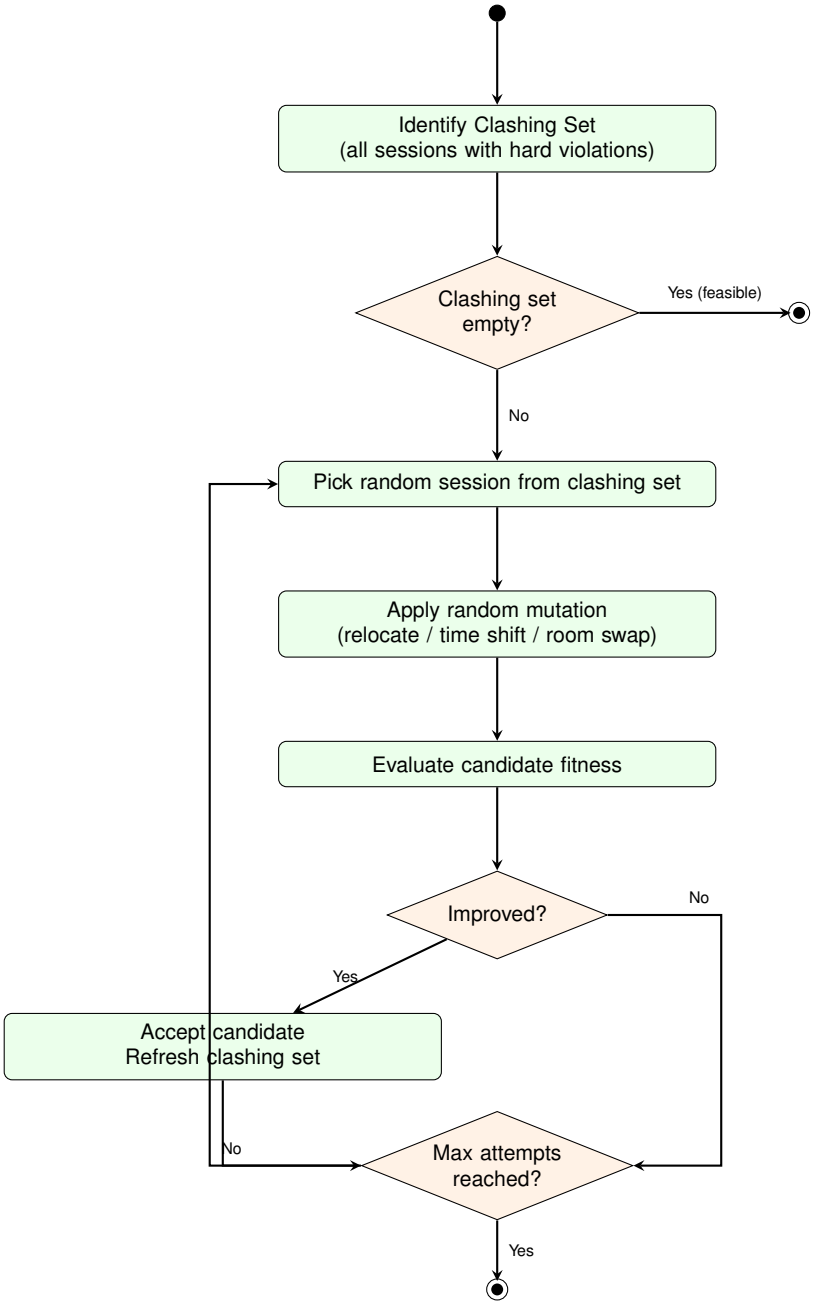

FIGURE 5.4: Activity Diagram: LNS Destroy-and-Repair Auto-Fix

5.9.1 Step 1: Cluster Selection

The administrator selects a `semester_cluster` (e.g., "IT, Sem 3, Batch 2023"). This scopes the scheduling problem by loading the associated subjects (with L-T-P configurations) and student groups (sections like Sec A–D, WMC, and IT-BI) from the database.

5.9.2 Step 2: Home Room Mapping

Each student group (section) is mapped to a home room. Home rooms follow IIITA's floor-based naming convention: `CC-3-5{floor}{suffix}`, where the floor digit is derived from the semester (semesters 1–2 $\rightarrow$ floor 0, semesters 3–4 $\rightarrow$ floor 1, semesters 5–6 $\rightarrow$ floor 2) and the suffix maps to the section letter (A$\rightarrow$06, B$\rightarrow$07, C$\rightarrow$54, D$\rightarrow$55). For example, Semester 3, Section B maps to room `CC-3-5107`. The wizard auto-applies this policy when a cluster is selected, falling back to round-robin assignment for any group that cannot be matched.

5.9.3 Step 3: Professor Assignment Matrix

An interactive matrix presents subjects as rows and student groups as columns. For each cell, the administrator selects a professor from a dropdown filtered by the `professor_expertise` table. Each subject supports four scheduling modes:

- **Sections:** Assigns individually to each section (A, B, C, D).
- **All (WMC):** Assigns to the whole-batch "WMC" group (used for core lectures taught to all sections simultaneously).
- **IT-BI:** Assigns only to the "IT-BI" group (for subjects exclusive to the IT-BI programme).

- **Disabled:** Excludes the subject entirely from the solver.

An “Auto-Fill” button populates all unset cells using round-robin professor assignment from the expertise pool.

5.9.4 Step 4: Seed and Lock Configuration

Before launching the solver, the administrator may optionally:

- Select a **seed timetable** to warm-start the solver from an existing schedule rather than a random initial solution.
- Select one or more **locked timetables** whose slots are injected as immutable constraints into the solver, enabling cross-semester conflict avoidance.

5.9.5 Step 5: Solver Execution and Results

Upon clicking “Generate Timetable,” the system constructs the `SolverInput`, launches the (1+1)-ES solver, and displays a real-time progress card showing: current generation, best fitness, σ, success rate, and the violation breakdown. Upon completion, a results panel displays the final fitness and provides options to:

- Save the generated timetable to the database as a new draft.
- Transition directly to the `EditorView` for manual refinement.
- Download a comprehensive diagnostic log file.

5.9.6 End-to-End Sequence Diagram

Figure 5.5 illustrates the complete interaction sequence between the administrator, the Intelligent Generator UI, the database, and the solver engine during an automated timetable

generation session.

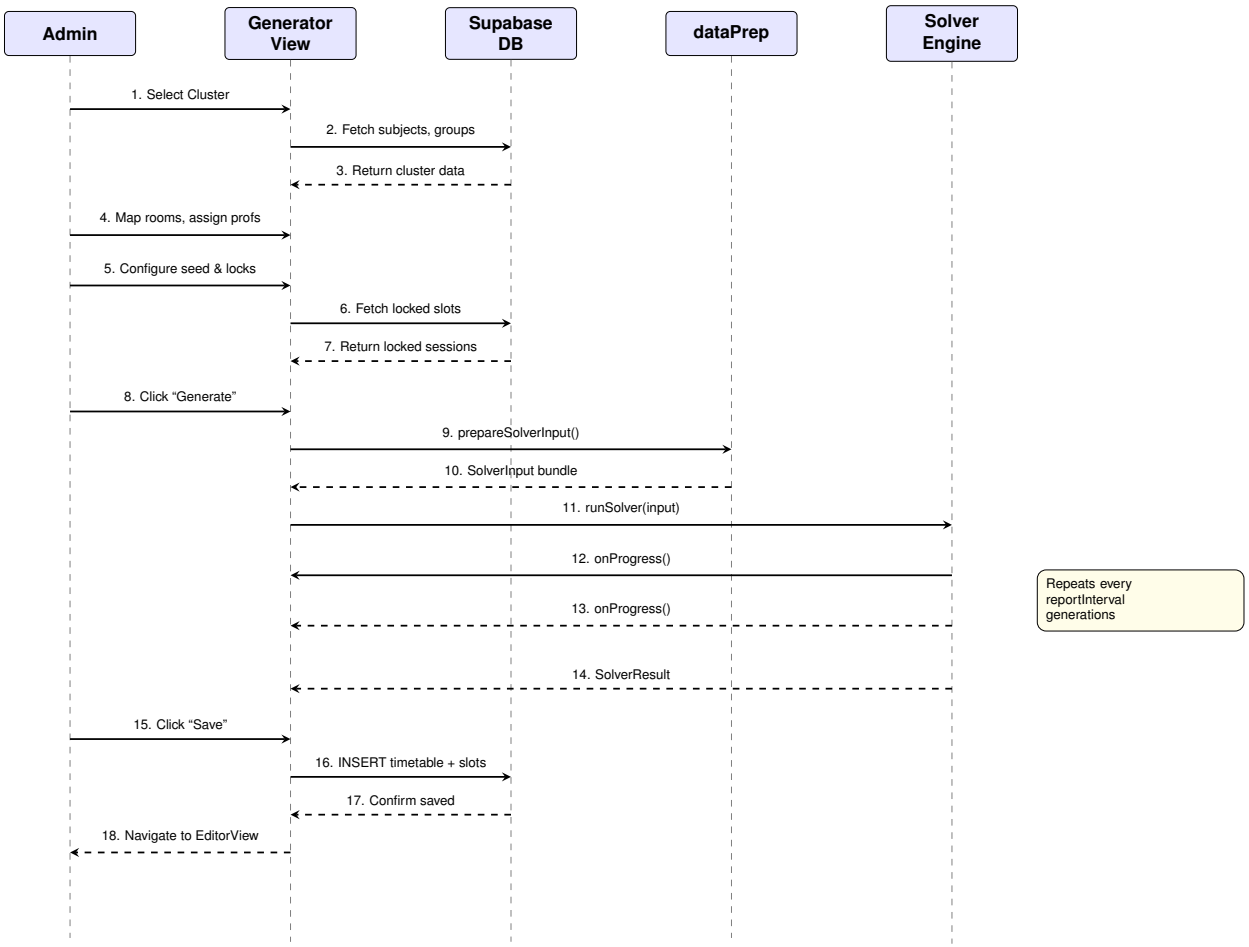

FIGURE 5.5: Sequence Diagram: End-to-End Intelligent Generator Workflow

Chapter 6

Conclusion and Future Work

Timetabling touches every person on a university campus, every single day. Getting it wrong means wasted hours, frustrated students, and overworked coordinators. In this thesis, we set out to replace the manual, spreadsheet-driven scheduling process at IIIT Allahabad with something better: a modern, web-based platform that can generate, edit, and distribute timetables intelligently.

6.1 Summary of Contributions

The biggest achievement of this project is taking what used to be a weeks-long manual process and turning it into something that runs in seconds. Here is what we built:

- **Heuristic Ingestion Engine:** We wrote an Excel parser that can read IIITA's messy, human-formatted spreadsheets, pull out the important bits (subjects, faculty, rooms), and load everything into a clean PostgreSQL database. This alone saves hours of manual data entry.
- **Real-Time Conflict Resolution:** The implementation of a multi-dimensional $O(N^2)$ clash detection algorithm that automatically surfaces Room, Professor, Section, and

WMC-Section overlaps. This feature alone significantly reduces the administrative burden of manually verifying schedule integrity.

- **(1+1)-ES Automated Solver:** The implementation of a fully functional timetable generator using a (1+1) Evolution Strategy algorithm. The solver evaluates candidate schedules against a comprehensive fitness function comprising ten hard constraints (room, professor, and group non-overlap; break/lunch boundary enforcement; lab-room assignment; elective basket synchronization; WMC-section isolation; home-room binding; the 2+1 lecture format; and time boundary compliance) and three soft constraints (student schedule gaps, room utilization efficiency, and professor same-half-day avoidance). The algorithm employs Rechenberg's 1/5th success rule for adaptive step-size control and a stagnation restart mechanism.

- **Interactive Timetable Editor:** The development of a drag-and-drop editing interface with real-time conflict visualization, inline room and professor editing, a 30-level undo stack, and a Large Neighbourhood Search (LNS) auto-fix routine that targets clashing sessions. The editor's Master View provides cross-semester constraint awareness by overlaying slots from all published timetables.

- **Multi-Step Intelligent Generator:** The creation of a guided configuration workflow enabling administrators to select semester clusters, auto-map home rooms using a floor-based naming policy, assign professors via an interactive matrix with expertise-based filtering, and configure seed and locked timetables before launching the solver.

- **Cross-Semester Constraint Awareness:** The implementation of locked sessions from published timetables, enabling the solver and editor to detect and avoid room and professor conflicts across independently scheduled semesters.

- **Stakeholder-Specific Interfaces:** The creation of dynamic, role-based visualization components. The dedicated “Student View” for filtering specific cohort schedules, the “Professor View,” and the macro-level “Master Occupancy” and “Free Room Finder” views ensure that every user receives actionable data without cognitive overload.
- **Modern Technical Architecture:** The deployment of a high-performance frontend utilizing React 19, TypeScript, and Vite, paired with a serverless Supabase backend. This stack ensures the application is highly responsive, strictly typed, and easily maintainable.
- **Professional Export Capabilities:** The integration of client-side PDF generation and automated Google Sheets API formatting, bridging the gap between dynamic web views and the necessity for static, offline documentation.

6.2 Current Limitations and Proposed Enhancements

No system is perfect, and ours is no exception. The limitations we have identified point directly to the most promising areas for future work. Table 6.1 summarises what we see as the key gaps and how they could be addressed.

6.3 Concluding Remarks

Looking back, we are proud of what we have built. The IIITA Timetabling System takes a problem that used to consume weeks of manual effort and solves it in seconds, right in the browser. Our (1+1)-ES solver reliably produces feasible, high-quality timetables; the drag-and-drop editor lets administrators fine-tune results with full constraint awareness; and

TABLE 6.1: System Limitations and Future Enhancements

Domain	Current Limitation	Proposed Enhancement
Data Ingestion	Heavily reliant on hardcoded regular expressions to parse IITA's specific Excel format.	Implementation of a dynamic mapping UI allowing users to visually map Excel columns to DB fields.
Solver Performance	Single-threaded (1+1)-ES running on the browser's main thread via batched `setTimeout`. Sufficient for current problem sizes but may become a bottleneck for larger institutions.	Migration to Web Workers for parallel solver execution, or a server-side solver for larger problem instances. Exploration of hybrid meta-heuristics (e.g., memetic algorithms combining ES with local search).
Solver Optimality	The Evolution Strategy is a meta-heuristic; solutions are feasible but not guaranteed to be globally optimal with respect to soft constraints.	Investigation of multi-objective optimization (Pareto-optimal fronts) and hybridization with constraint programming (CP) solvers for provable optimality on subproblems.
State Management	Relies on localized React state and custom hooks, potentially causing prop-drilling in deep components.	Adoption of a global state manager (e.g., Zustand or Redux) to optimize data sharing across views.
Notifications	Users must actively check the web portal for schedule updates.	Development of a Progressive Web App (PWA) with Push Notifications via Firebase Cloud Messaging.
Analytics	Basic occupancy visualization without long-term historical analysis.	Dedicated BI Dashboard to track room utilization metrics across multiple academic years.

the role-specific views ensure that every stakeholder—from first-year students to campus administrators—gets exactly the information they need. Perhaps most importantly, the project demonstrates that you do not need heavyweight server infrastructure to solve real scheduling problems: a well-designed, pure-TypeScript solver running on the client side is more than enough for a real-world university. We hope this work serves as a practical blueprint for other institutions looking to modernise their scheduling workflows.

Appendix A

Core Algorithm Snippets

This appendix provides crucial code excerpts from the React 19 codebase, demonstrating the practical implementation of the theoretical concepts discussed in the main chapters.

A.1 Dynamic Grid Generation Logic

The following TypeScript snippet from the `TimetableViewer` component demonstrates how the system extracts unique time boundaries to dynamically generate responsive column headers, preventing the display of empty, unused time blocks.

LISTING A.1: Dynamic Time Column Extraction Algorithm

```
// Extract all unique start and end times from the active slots
const allTimes = useMemo(() => {
  const times = new Set<string>();

  // Inject hardcoded institutional break times
  times.add("10:50");
  times.add("11:00");
  times.add("13:00");
  times.add("14:30");

  // Extract times from all valid timetable slots
  slots.forEach(slot => {
    if (slot.start_time && slot.end_time) {
      times.add(slot.start_time.substring(0, 5));
      times.add(slot.end_time.substring(0, 5));
    }
  });

  // Sort chronologically and format as standard columns
  return Array.from(times).sort().reduce((acc, curr, index, arr) => {
    if (index < arr.length - 1) {
      acc.push(`${curr}-${arr[index + 1]}`);
    }
    return acc;
  }, [] as string[]);
}, [slots]);
```

A.2 Real-Time Clash Detection Filter

This snippet illustrates the $O(N^2)$ comparison logic used to immediately identify Room, Professor, and Section overlaps within the currently selected semester block.

LISTING A.2: Overlap Detection and Categorization Logic

```
// O(N^2) comparison to find all overlapping events
const newClashes = filteredSlots.filter((slot, i) => {
  return filteredSlots.some((otherSlot, j) => {
    // Skip self-comparison
    if (i === j) return false;
    // Must be on the same day to clash
    if (slot.day_of_week !== otherSlot.day_of_week) return false;

    // Check for temporal intersection
    const isTimeOverlap = slot.start_time < otherSlot.end_time &&
                          slot.end_time > otherSlot.start_time;

    if (isTimeOverlap) {
      // 1. Room Clash
      if (slot.rooms?.name === otherSlot.rooms?.name) return true;
      // 2. Professor Clash
      if (slot.professors?.name === otherSlot.professors?.name) return
          true;
      // 3. Section Clash (Same section, different subject)
      if (slot.student_groups?.name === otherSlot.student_groups?.name &&
          slot.subjects?.id !== otherSlot.subjects?.id) {
          return true;
      }
    }
    return false;
  });
});
```

A.3 Solver Fitness Evaluation Function

The following excerpt from `constraints.ts` demonstrates the combined fitness evaluation that scores a candidate timetable against all ten hard constraints and three soft constraints.

LISTING A.3: Combined Fitness Evaluation (constraints.ts)

```
export function evaluate(input: SolverInput, solution: Solution):
   FitnessResult {
    const { sessions, rooms, numDays, config } = input;

    // Count all hard constraint violations
    const timeBoundary = countTimeBoundaryViolations(sessions, solution);
    const breakCrossing = countBreakViolations(sessions, solution);
    const roomOverlap = countRoomOverlaps(sessions, solution, rooms);
    const professorOverlap = countProfessorOverlaps(sessions, solution);
    const groupOverlap = countGroupOverlaps(sessions, solution);
    const electiveSync = countElectiveSyncViolations(sessions, solution);
    const labRoom = countLabRoomViolations(sessions, solution, rooms);
    const wmcSectionOverlap = countWMCSectionOverlaps(sessions, solution);
    const homeRoom = countHomeRoomViolations(sessions, solution);
    const twoOneLecture = countTwoOneLectureViolations(sessions, solution)
       ;

    const hardViolations = timeBoundary + breakCrossing + roomOverlap +
        professorOverlap + groupOverlap + electiveSync + labRoom +
        wmcSectionOverlap + homeRoom + twoOneLecture;
```

```
    // Compute soft constraint penalties
    const gapPenalty = computeGapPenalty(sessions, solution, numDays);
    const roomUtilizationPenalty = computeRoomUtilizationPenalty(
        sessions, solution, rooms,
        config.roomUtilizationThreshold,
        config.roomOverutilizationThreshold,
    );
    const professorSameHalfPenalty = computeProfessorSameHalfPenalty(
        sessions, solution, numDays
    );

    // Weighted composite fitness
    const total = hardViolations * config.hardPenalty
        + gapPenalty * config.gapWeight
        + roomUtilizationPenalty * config.roomUtilizationWeight
        + professorSameHalfPenalty * config.professorSameHalfWeight;

    return { total, hardViolations, gapPenalty,
        roomUtilizationPenalty, professorSameHalfPenalty,
        violationBreakdown: { timeBoundary, breakCrossing,
            roomOverlap, professorOverlap, groupOverlap,
            electiveSync, labRoom, wmcSectionOverlap,
            homeRoom, twoOneLecture },
    };
}
```

A.4 (1+1)-ES Solver Main Loop

The core optimization loop from `solver.ts`, showing elitist selection, σ-adaptation via the 1/5th success rule, and stagnation restart.

LISTING A.4: (1+1)-ES Main Loop (solver.ts)

```
export async function runSolver(
    input: SolverInput,
    onProgress?: (p: SolverProgress) => void,
    cancelToken?: { cancelled: boolean },
    seedSolution?: Solution,
    lockedGenes?: Gene[],
): Promise<SolverResult> {
    const { config } = input;
    let parent = seedSolution
        ? seedSolution.map(g => ({ ...g }))
        : generateInitialSolution(input);

    // Pin locked genes from published timetables
    if (lockedGenes) {
        const offset = input.sessions.length - lockedGenes.length;
        for (let i = 0; i < lockedGenes.length; i++) {
            parent[offset + i] = { ...lockedGenes[i] };
        }
    }

    let bestFitness = evaluate(input, parent);
    let bestSolution = parent.map(g => ({ ...g }));
    let sigma = config.initialSigma;
```

```
    let successes = 0;

    for (let gen = 1; gen <= config.maxGenerations; gen++) {
        if (cancelToken?.cancelled) break;

        const offspring = mutate(parent, input, sigma);
        // Re-pin locked genes (mutations must not move them)
        if (lockedGenes) { /* ... pin logic ... */ }
        const offFitness = evaluate(input, offspring);

        // Elitist selection: accept if offspring is <= parent
        if (offFitness.total <= bestFitness.total) {
            parent = offspring;
            bestFitness = offFitness;
            bestSolution = offspring.map(g => ({ ...g }));
            successes++;
        }

        // 1/5th rule sigma adaptation every W generations
        if (gen % config.adaptationWindow === 0) {
            const rate = successes / config.adaptationWindow;
            if (rate > 0.2) sigma *= config.sigmaIncrease;
            else if (rate < 0.2) sigma *= config.sigmaDecrease;
            successes = 0;
        }

        // Stagnation restart
        if (stagnationCounter > stagnationThreshold) {
            parent = generateInitialSolution(input);
            // Re-pin locked genes ...
        }

        // Early termination if feasible
        if (bestFitness.hardViolations === 0) break;
    }
    return { solution: bestSolution, fitness: bestFitness, ... };
}
```

A.5 LNS Auto-Fix for the Editor

The Large Neighbourhood Search routine from `localSearch.ts`, which identifies all clashing sessions and iteratively repairs them.

LISTING A.5: Full LNS Destroy-and-Repair (localSearch.ts)

```
export function runFullLNS(
    input: SolverInput,
    solution: Solution,
    maxAttempts: number = 1000,
): { solution: Solution; fitness: FitnessResult } | null {
    const baseFitness = evaluate(input, solution);
    if (baseFitness.hardViolations === 0) {
        return { solution, fitness: baseFitness };
    }

    // Find all session indices involved in violations
    let clashingIndices = findClashingIndices(input, solution);
```

```
    let targets = Array.from(clashingIndices);
    let bestSolution = solution.map(g => ({ ...g }));
    let bestFitness = baseFitness;

    for (let attempt = 0; attempt < maxAttempts; attempt++) {
        const candidate = bestSolution.map(g => ({ ...g }));
        // Pick a random clashing session
        const idx = targets[Math.floor(Math.random() * targets.length)];
        const session = input.sessions[idx];

        // Apply random mutation: relocate (50%), time shift (30%), room
           (20%)
        const strategy = Math.random();
        if (strategy < 0.5)
            candidate[idx] = mutateRelocate(session, input.rooms);
        else if (strategy < 0.8)
            candidate[idx] = { ...candidate[idx],
                ...mutateTime(candidate[idx], session, 3.0) };
        else
            candidate[idx] = { ...candidate[idx],
                roomIndex: mutateRoom(session, input.rooms).roomIndex };

        const candidateFitness = evaluate(input, candidate);
        if (candidateFitness.total < bestFitness.total) {
            bestSolution = candidate;
            bestFitness = candidateFitness;
            if (bestFitness.hardViolations === 0) break;
            // Refresh targets after improvement
            clashingIndices = findClashingIndices(input, bestSolution);
            targets = Array.from(clashingIndices);
        }
    }
    return bestFitness.total < baseFitness.total
        ? { solution: bestSolution, fitness: bestFitness } : null;
}
```